

PY101: Week 3 - Basic Data Structures

Lesson 1: String Skills Upgrade I: Indexing and Slicing

In Python, string positions start counting at 0 for positive indexing and -1 when counting backward from the end of a string. Slicing follows the format `[start:stop:step]`, where the `start` index is inclusive and the `stop` index is strictly exclusive.

```
word = "python"
print(word[1:4])    # yth
print(word[::-1])   # nohtyp (reversed)
```

The 'Stop' Exclusion Trick

Because the `stop` parameter is exclusive, the length of the slice will always be equal to `stop - start`. For example, `word[1:4]` results in exactly $4 - 1 = 3$ characters ("yth"). This makes calculating slice lengths fast and intuitive.

Lesson 2: String Skills Upgrade II: Common Methods

Python strings provide powerful built-in methods to clean, format, and search text. Methods like `.strip()` remove trailing and leading whitespace, `.lower()` normalizes character casing, `.replace()` substitutes substrings, and `.find()` tracks structural matching by returning indices or -1 when an item is missing.

```
cleaned = "  SpaM ".strip().lower()    # "spam"
position = "hello".find("l")            # 2
```

Immutability Warning

String methods do not alter the original string! Strings in Python are **immutable**. When you call `word.lower()`, Python generates a brand-new string in memory. If you want to keep the changes, you must reassign them back to a variable.

Lesson 3: Expressions and Operator Precedence

Python calculates evaluations based on standard mathematical operator precedence (BEDMAS/PEMDAS). Parentheses explicitly override default priorities, forcing grouped operations to execute first. Standard division (`/`) in Python always evaluates to a `float` type.

```
result = 2 + 3 * 4    # 14 (multiplication first)
grouped = (2 + 3) * 4 # 20 (parentheses first)
```

Why `6 / 2` Equals `3.0`

In Python 3, the single forward slash `/` is the true division operator and is designed to always maintain maximum fractional precision by outputting a float. If you explicitly require an integer output without a decimal, use the floor division operator (`//`) instead.

Lesson 4: Lists I: Creating Lists

Lists are ordered collections enclosed in square brackets (`[]`). They can store multiple elements as single, manageable objects, hold multiple structural items generated sequentially from `list(range())`, and dynamically modify their elements because they are mutable data types.

```
foods = ["pizza", "ramen", "salad"]
numbers = list(range(1, 6)) # [1, 2, 3, 4, 5]
```

List Literals vs. Separate Variables

Grouping related records into a single list element allows you to scale your program easily. Instead of tracking separate items via `temp1`, `temp2`, `temp3`, a list literal lets you use structural collection functions like `len()` or run iteration loops over the data smoothly.

Lesson 5: Lists II: Indexing and Slicing

List indexing matches string positioning behaviors, using `0` for the first position and `-1` for the final tail element. Slice boundaries can be omitted to span elements to the borders or utilize negative step sizes to manipulate structural order efficiently.

```
temps = [65, 68, 70, 69, 71]
middle = temps[1:-1]    # [68, 70, 69]
temps[-1] += 2          # Modifies index directly
```

Catching `IndexError` Bugs

Attempting to access an index position outside the length boundaries of a list (like `list[99]` on a list of length 3) triggers an `IndexError`. Always ensure your index evaluations fall safely between `0` and `len(list) - 1`.

Lesson 6: Lists III: Mastering List Iteration

Python offers two ways to loop through a list: index-based loops using `range(len(list))`, which track sequential index locations, and direct value loops, which process matching elements sequentially without calculating positions.

```
# Index loop
for i in range(len(snacks)):
    print(i, snacks[i])

# Value loop
for item in snacks:
    print(item)
```

When to use Index Loops vs. Value Loops

Choose direct value loops by default because they are cleaner and more succinct. You only need an index-based loop when your script specifically requires knowing or modifying the element's exact location, matching paired positions across multiple lists, or altering data rows in place.

Lesson 7: Lists IV: Common Methods

List manipulation relies on specific built-in methods: `.append()` inserts a singular value at the tail, `.insert()` maps items to exact index targets, `.remove()` searches out and destroys the first matching value, and `.pop()` extracts and returns an item from the list.

```
shopping.append("Milk")
shopping.insert(0, "Apple")
if "spam" in shopping:
    shopping.remove("spam")
```

The Critical Difference: `.append()` vs. `.extend()`

The `.extend()` method expects a collection and merges its items one by one into the target list. The `.append()` method inserts whatever object it is given as a single item. Appending a list inside another list creates a nested structure (a list inside a list), while extending keeps the list flat.

Lesson 8: Ordering Lists: `sorted()` vs. `.sort()`

The `.sort()` method organizes elements in place, directly modifying the source collection while returning `None`. Conversely, the `sorted()` function extracts elements to construct a separate, ordered duplicate, leaving the source material untouched.

```
prices.sort()                # Modifies prices in place
cheap_first = sorted(prices) # Generates an ordered copy
```

What is Sort Stability?

Python uses a sorting algorithm called Timsort, which is **stable**. This means if two elements have identical sorting keys (for example, sorting words by length when two words have 4 letters), they will keep their original relative order from the source list, preventing unexpected re-ordering bugs.

Lesson 9: Dictionaries I: Keys and Values

Dictionaries store information in associated key-value pairs inside curly braces (`{}`). Instead of using numeric position indices, dictionaries look up values directly by their unique keys. Dictionary keys must use immutable data types, such as strings, integers, or tuples.

```
ages = {"bob": 42, "maya": 35}
print(ages["bob"])    # Looks up with the key name
ages["lucas"] = 9     # Appends new entry smoothly
```

Why Can't Lists Be Dictionary Keys?

Python requires dictionary keys to be **hashable**, meaning their identity value can never change. Because lists are mutable (they can be modified after creation), they are unhashable and will crash your program with a `TypeError` if used as a key. Tuples, being immutable, are safe to use.

Lesson 10: Dictionaries II: Iteration Patterns

Loop structures can target different parts of a dictionary: the default loop handles structural keys, `.values()` processes matching data rows sequentially, and `.items()` unpacks both keys and values simultaneously into paired loop variables.

```
for name, age in ages.items():  
    print(f"{name} is {age}")
```

Handling Missing Keys Safely with `.get()`

Looking up a key that doesn't exist using square brackets (like `dict["missing"]`) causes a `KeyError` crash. Using the `.get()` method allows you to safely read keys by returning a default fallback value (like `None` or `0`) instead of crashing if the key is missing.

Lesson 11: Updates and Copy Semantics

Simple reference assignment (`a = b`) creates an **alias** in memory, meaning both variable names point to the exact same underlying object. To safely make independent duplicates that can be modified without altering the source data, you must perform a copy operation.

```
working = nums.copy()          # Safe list duplicate  
dict_backup = prices.copy()    # Safe dictionary duplicate
```

The Identity 'is' Operator vs. '=='

The `==` operator evaluates whether two variables have the same **value**. The `is` operator evaluates whether two variables point to the exact same **object in memory** (identity). If you copy a list, `==` will be `True`, but `is` will be `False`.

Lesson 12: Nested Structures and Safe Mutation

When lists or dictionaries are nested inside one another, a standard shallow copy operation (`.copy()`) only copies the outermost container. The inner data paths remain linked as aliases, meaning changes made to nested items will affect both variables unless the inner elements are also copied.

```
b = a.copy()
b[0] = a[0].copy()    # Break structural inner alias links
b[0][1] = 99          # Safe deep mutation
```

Understanding Shallow Copies vs. Deep Copies

A shallow copy creates a new outer container but populates it with references to the original inner objects. To completely isolate complex nested structures without manually duplicating every inner row, you can import Python's built-in `copy` module and use `copy.deepcopy()`.

Lesson 13: Parsing Text to List of Dicts: Tokenize and Map Fields

Raw text data streams can be processed into highly organized data structures using a systematic pipeline: `.split()` tokenizes comma-separated rows into lists, `.strip()` removes extra whitespace, and structural fields are mapped to named dictionary keys before being appended to a collector list.

```
for line in lines:
    fields = line.split(",")
    record = {"name": fields[0].strip(), "age": int(fields[1])}
    records.append(record)
```

The Power of Structured Records

Converting plain unformatted text rows into a list of structured dictionaries makes complex operations easy. Once data is stored in this format, you can easily sort records by specific values, filter rows using condition checks, or uniquely identify individual entries using database-style IDs.